# ogGIT

A Fully Decentralized, Peer-to-Peer Git

Complete Implementation Guide (Rust)

Design and Engineering Handbook

October 2026 — Version 0.1

**Abstract**

This document is a complete, buildable plan for ogGIT: a Git-compatible code collaboration system with no central server. Developers identify themselves with cryptographic keys, repositories are content-addressed and replicated peer-to-peer, small "developer circles" control who may publish to a project, and everything works through the ordinary `git` command line. The guide covers requirements, architecture, data model, network protocol, access control, collaboration objects, Git integration, performance engineering, security, testing, and a phased roadmap, together with Rust code skeletons for the critical components.

## Contents

**1 Vision, Goals and Scope** **4**

1.1 What ogGIT is . . . 4

1.2 Requirements and how the design satisfies them . . . 4

1.3 Non-goals for version 1 . . . 4

**2 Reality Check and Prior Art** **5**

2.1 Prior art . . . 5

2.2 The genuinely hard problems . . . 5

**3 Architecture** **6**

3.1 Layered view . . . 6

3.2 Processes and binaries . . . 6

3.3 Key design decisions . . . 6

3.4 Life of a push . . . 7

**4 Technology Stack** **7**

**5 Data Model** **8**

5.1 Identities and keys . . . 8

5.2 Repository identity (genesis) . . . 9

5.3 On-disk layout . . . 10

5.4 Signed ref snapshots (sigrefs) . . . 10

5.5 Circle log . . . 11
5.6 Petnames . . . 11

**6 Network Layer 11**
6.1 Transport stack . . . 11
6.2 Discovery layers (from nearest to farthest) . . . 11
6.3 NAT traversal . . . 12
6.4 Gossip . . . 12
6.5 Libp2p behaviour sketch . . . 12

**7 Sync and Replication Engine 13**
7.1 Two-stage strategy . . . 13
7.2 Stage A details . . . 13
7.3 Sync loop . . . 13
7.4 Stage B: multi-peer parallel fetch . . . 13
7.5 Large Object Store (LOS) . . . 14
7.6 Seeding policy . . . 14

**8 Developer Circles and Access Control 14**
8.1 Roles . . . 15
8.2 Invite flow (target: under 60 seconds) . . . 15
8.3 Canonical branches by quorum . . . 15
8.4 Private repositories . . . 16
8.5 Key rotation and recovery . . . 16

**9 Collaboration Objects: Issues, Patches, Reviews 16**

**10 Git Integration 17**
10.1 Remote helper . . . 17
10.2 URL forms . . . 18
10.3 Compatibility extras . . . 18

**11 CLI and User Experience 18**
11.1 Sixty-second tour . . . 18
11.2 Command reference (v1) . . . 19
11.3 Usability rules . . . 19

**12 Daemon Internals 19**
12.1 Actor structure . . . 19
12.2 Rules for staying fast and correct . . . 20
12.3 Local APIs and UI . . . 20

**13 Performance Engineering 21**

13.1 Where time goes in a clone . . . 21
13.2 Resource budgets (targets to validate, not promises) . . . 21
13.3 Benchmarking plan . . . 21

**14 Security and Threat Model** **22**

**15 Code Skeleton** **23**
15.1 Workspace layout . . . 23
15.2 Workspace manifest . . . 23
15.3 Build order inside the code . . . 24

**16 Testing and Quality** **24**
16.1 Invariants to assert everywhere . . . 24

**17 Roadmap** **25**

**18 Risks and Open Questions** **25**

**A Wire Protocol Summary** **26**
A.1 Protocol identifiers . . . 26
A.2 Frame format for control messages . . . 26
A.3 Error codes . . . 26

**B Example Configuration** **26**

**C Week-One Spike (do this first)** **27**

**D References and Further Reading** **27**

# 1 Vision, Goals and Scope

## 1.1 What ogGIT is

ogGIT keeps Git's best idea (an immutable, content-addressed object graph) and replaces the part that is centralized in practice: the hosting server, the account system, and the permission model. A repository in ogGIT is not "a place"; it is a *set of signed data* that any willing peer can store and serve.

## 1.2 Requirements and how the design satisfies them

| # | Requirement | Design answer | How we verify |
|---|---|---|---|
| 1 | Completely decentralized | No privileged server. Identity = Ed25519 keys. Discovery = DHT + gossip + LAN multicast. Any node may seed. | Kill any node (including bootstrap nodes); existing swarms keep working. |
| 2 | P2P, Freenet-like | Content addressing, key-based routing (Kademlia), replication driven by interest, optional anonymity transports later. | Network partition and churn simulation. |
| 3 | Fast upload/download | QUIC, long-lived daemon, packfiles with deltas, reachability bitmaps, multi-peer parallel fetch, chunked large-object store. | Benchmarks vs. a hosted Git service on the same link (Section 13.3). |
| 4 | Easy developer circle | Signed membership log, roles, one-line invite links, follow lists, quorum-controlled canonical branches. | Onboard a new member in under 60 seconds (user test). |
| 5 | Seamless and super fast, Rust/C++ | 100% Rust (memory safety on untrusted network input), async I/O, pure-Rust Git via `gitoxide`. | Cold-start latency, throughput, and memory budgets in CI. |
| 6 | Easy to set up and access | Single static binary, zero-config `og init`, LAN auto-discovery, invite links. | Install-to-first-clone time. |
| 7 | Very helpful to developers | Works with plain `git`; offline-first issues and patches; TUI and local web UI; CI hooks. | Dogfood: ogGIT is developed on ogGIT. |

## 1.3 Non-goals for version 1

- A global, human-readable namespace ("github.com/user/repo"). Names are local aliases; see Section 5.6.
- Forced anonymity. Anonymity transports (Tor, I2P) are a later add-on.

- Replacing Git's object format. ogGIT *extends around* Git and stays 100% Git compatible.
- A blockchain or token. Nothing in this design requires global consensus.

# 2 Reality Check and Prior Art

Before writing code, be clear about what exists and what is genuinely hard.

## 2.1 Prior art

| System | Idea | Lesson for ogGIT |
|---|---|---|
| Radicle | P2P code collaboration on top of Git; Ed25519 identities, signed refs, gossip replication, seed nodes, collaborative objects for issues and patches. | Closest relative. Study its design first; many of the choices below are in the same family. |
| git-ssb | Git over Secure Scuttlebutt (gossip-based feeds). | Social-graph replication works, but whole-feed replication is heavy. |
| git-remote-ipfs | Git objects stored in IPFS. | Content addressing is easy; mutable refs, access control and speed are the hard parts. |
| Blockchain Git hosts | Refs or ownership recorded on a chain. | Global consensus is unnecessary for Git and adds cost and latency. |
| Freenet | Small-world key-based routing, content stored by locality of interest. | Inspires routing and replication-by-interest; its anonymity-first design costs speed. |

**Honest advice before you start**

If your only goal is a working decentralized Git, evaluate Radicle first. Building ogGIT is worthwhile if you want its differentiators: (a) first-class *developer circles* with invite links and quorum-controlled branches, (b) multi-peer swarm downloading and a chunked large-object store, (c) a single daemon that is excellent on LANs and offline, and (d) full ownership of the stack. This guide is written assuming you have decided to build.

## 2.2 The genuinely hard problems

1. **Availability.** Data exists only while someone stores it. Solution: opt-in always-on *seed nodes* and pinning, plus the circle seeding by default.
2. **Discovery and bootstrapping.** A new node must find peers. Solution: invite links carry seed hints; configurable bootstrap list; mDNS on LAN.
3. **Revocation.** You cannot un-share data. Solution: revocation stops *future* access and rotates keys; document the limit clearly.
4. **Spam and Sybil attacks.** Anyone can create peers. Solution: trust is *repo-scoped*: only data signed by circle members affects canonical state.
5. **Names.** Decentralized, secure and human-memorable names cannot all hold at once (Zooko's triangle). Solution: cryptographic IDs plus local petnames.

# 3 Architecture

## 3.1 Layered view

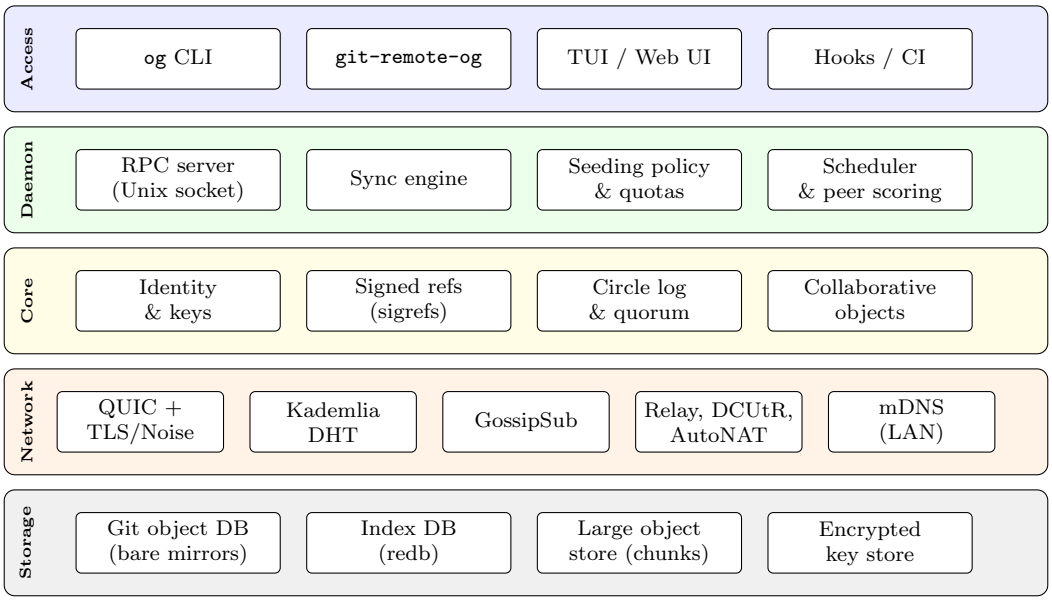



Figure 1: ogGIT layers. Everything below the access layer lives in one long-running process, `ogd`.

## 3.2 Processes and binaries

| Binary | Role |
|---|---|
| `og` | User-facing CLI (init, clone, circle, issue, patch, node, status). Thin client of `ogd`. |
| `ogd` | The node: networking, sync, storage, policy. Starts on demand (socket activation) or as a system service. |
| `git-remote-og` | Git *remote helper*. Lets ordinary `git clone og://...`, `git push og`, `git pull og` work. |

## 3.3 Key design decisions

**Decisions that shape everything else**

1. **Git stays the object store.** Commits, trees and blobs are normal Git objects in normal bare repositories. Any Git tool can read them.
2. **Per-author namespaces.** Every contributor owns a namespace of refs; nobody can overwrite someone else's refs. This removes write conflicts by construction.
3. **Signed, append-only ref snapshots.** Each author publishes a signed chain of their ref state (*sigrefs*). Replication verifies signatures, sequence numbers and ancestry.
4. **Canonical branches by quorum.** The "official" `main` of a project is computed from the delegates' namespaces using a threshold rule.
5. **Local mirror first.** The user's `git` talks to a *local* mirror (fast, offline-capable); `ogd` syncs mirrors with peers in the background.

### 3.4 Life of a push

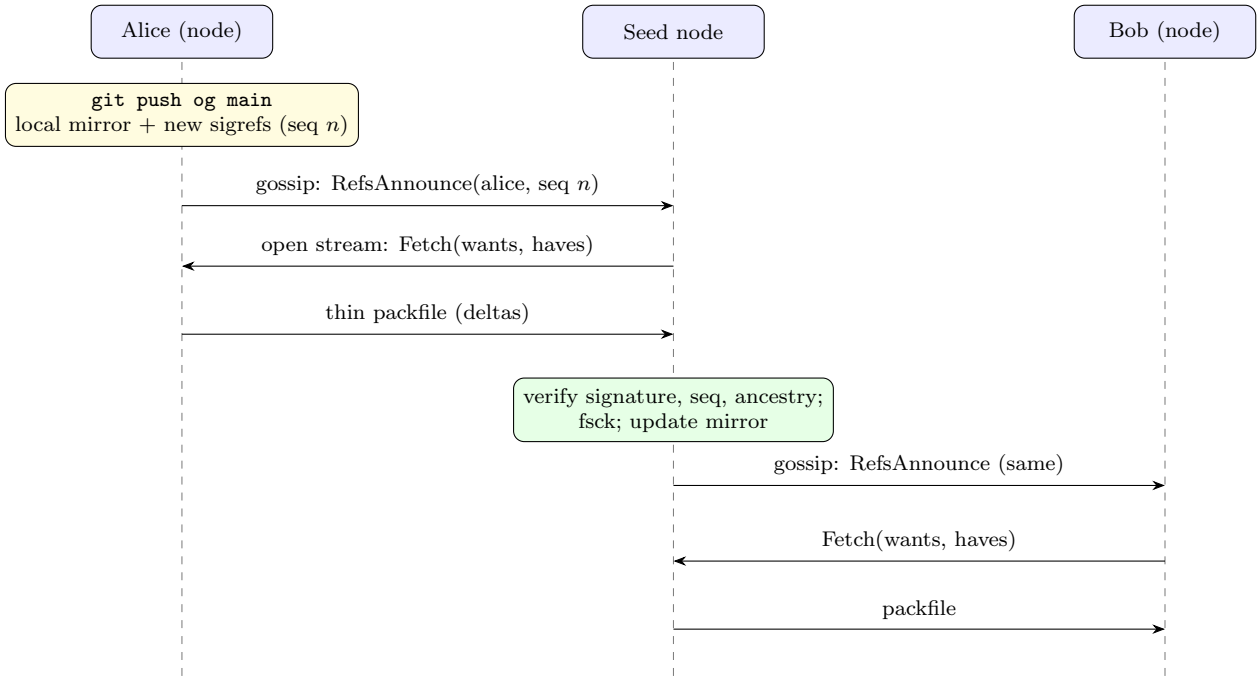



Figure 2: Replication of one push through a seed. Bob may fetch from Alice directly or in parallel from several peers.

## 4 Technology Stack

Rust is the right choice: the node parses untrusted binary input from strangers all day, so memory safety is a security property, not a luxury. C++ offers no speed advantage here, because the bottlenecks are network and disk. If you need a C/C++ library, wrap it behind a narrow FFI boundary.

| Concern | Choice (and why) |
|---|---|
| Async runtime | `tokio`. Standard, mature, works with libp2p and QUIC. |
| Git access | `gix` (gitoxide) for reading refs, objects and graph walks. Shell out to the `git` binary for pack generation, `index-pack`, `repack`, and bitmaps in version 1; replace gradually. |
| P2P networking | `libp2p` (QUIC, Noise, Kademlia, GossipSub, relay v2, DCUtR, AutoNAT, mDNS). Alternative: `iroh` (QUIC with built-in hole punching and relays, BLAKE3 verified streaming); a good choice if you prefer a smaller surface and handle discovery yourself. |
| Signatures | `ed25519-dalek` with `verify_strict`. |
| Hashing | `blake3` for protocol-level hashes and chunk addressing. Git objects keep Git's own IDs (SHA-1 with collision detection, SHA-256 when tooling is ready). |
| Serialization | `postcard` or `ciborium` (CBOR) for wire messages; canonical encoding for anything that is signed. |
| Index database | `redb` (pure Rust, ACID, simple). |
| Chunking | FastCDC (`fastcdc` crate) for large files. |
| CRDTs | `automerge` or a small custom op-based CRDT for issues and patches. |
| Encryption | `chacha20poly1305` (XChaCha), `x25519-dalek`, `argon2` for key-file passphrases. |
| CLI | `clap`, `ratatui` (TUI), `indicatif` (progress). |
| Web UI | `axum` serving a static single-page app on `127.0.0.1`. |
| Testing | `proptest`, `cargo-fuzz`, `libp2p-swarm-test`, `criterion`, network namespaces with `tc netem`. |

**On version numbers and APIs**

Crate APIs (especially libp2p and gitoxide) change between releases. The code in this guide is an architectural sketch, not copy-paste-verified code. Pin versions at project start and adapt the calls to the versions you choose.

# 5 Data Model

## 5.1 Identities and keys

Two key types keep daily life simple and allow multiple devices:

- **Author key** (Ed25519). Identifies a developer. Signs ref snapshots, circle entries, and collaborative objects. Displayed as a `did:key` style string.
- **Node key** (Ed25519). Identifies one machine on the network (libp2p PeerId). A *delegation certificate* signed by the author key says "this node acts for this author". A developer can run many nodes; losing a laptop revokes only that node.

Key storage: a passphrase-protected file (Argon2id to derive a key, XChaCha20-Poly1305 to encrypt). Later versions integrate with `ssh-agent`, OS keychains and FIDO2 hardware keys.

```
// og-core/src/id.rs
use ed25519_dalek::{Signature, Signer, SigningKey, Verifier, VerifyingKey};
use serde::{Deserialize, Serialize};

#[derive(Clone, Copy, PartialEq, Eq, Hash, PartialOrd, Ord, Serialize, Deserialize)]
pub struct AuthorId(pub [u8; 32]);   // Ed25519 public key
```

```rust
#[derive(Clone, Copy, PartialEq, Eq, Hash, Serialize, Deserialize)]
pub struct RepoId(pub [u8; 32]);     // BLAKE3 of the canonical genesis document

pub type Oid = [u8; 20];             // Git object id (SHA-1 in v1)

pub struct Author { sk: SigningKey }

impl Author {
    pub fn generate() -> Self {
        Self { sk: SigningKey::generate(&mut rand::rngs::OsRng) }
    }
    pub fn id(&self) -> AuthorId {
        AuthorId(self.sk.verifying_key().to_bytes())
    }
    /// Every signature is bound to a purpose string so a signature made for
    /// one message type can never be replayed as another.
    pub fn sign(&self, domain: &[u8], payload: &[u8]) -> Signature {
        let mut m = Vec::with_capacity(domain.len() + payload.len());
        m.extend_from_slice(domain);
        m.extend_from_slice(payload);
        self.sk.sign(&m)
    }
}

pub fn verify(id: &AuthorId, domain: &[u8], payload: &[u8], sig: &Signature) -> bool {
    let Ok(vk) = VerifyingKey::from_bytes(&id.0) else { return false };
    let mut m = Vec::with_capacity(domain.len() + payload.len());
    m.extend_from_slice(domain);
    m.extend_from_slice(payload);
    vk.verify_strict(&m, sig).is_ok()
}
```

Listing 1: Core identifiers and signing with domain separation.

## 5.2 Repository identity (genesis)

A repository is identified by the hash of its *genesis document*. The genesis lists the initial delegates and the threshold. Because the ID is a hash of this content, nobody can change who the original delegates were without creating a different repository.

```rust
#[derive(Serialize, Deserialize)]
pub enum Visibility { Public, Private }

#[derive(Serialize, Deserialize)]
pub struct Genesis {
    pub version: u8,
    pub name: String,            // a suggestion only, not a global name
    pub description: String,
    pub default_branch: String,  // e.g. "main"
    pub delegates: Vec<AuthorId>,
    pub threshold: u8,           // must be > delegates.len() / 2 when delegates > 1
    pub visibility: Visibility,
    pub nonce: [u8; 16],         // makes two identical-looking repos distinct
}

impl Genesis {
    pub fn repo_id(&self) -> RepoId {
        let mut buf = Vec::new();
        ciborium::into_writer(self, &mut buf).expect("serialize");
        RepoId(*blake3::hash(&buf).as_bytes())
    }
```

```
}
```

Listing 2: Genesis document and RepoId derivation.

### 5.3 On-disk layout

```
~/.og/
  config.toml                    # node configuration
  keys/author.key                # encrypted author key
  keys/node.key                  # node (transport) key
  index.redb                     # peers, providers, seen seq numbers, quotas
  run/ogd.sock                   # RPC socket for og and git-remote-og
  los/                           # large object chunks (content addressed)
  repos/<repo-id>/               # one BARE git repository per project
      refs/namespaces/<author>/refs/heads/...      # per-author branches
      refs/namespaces/<author>/refs/og/sigrefs     # signed snapshot chain
      refs/og/circle                               # circle log (a ref chain)
      refs/og/cobs/<type>/<id>                     # issues, patches, comments
```

Listing 3: Local storage layout.

Git's native *namespace* feature (`refs/namespaces/<name>/...`) already provides this layout, so standard tooling understands it.

### 5.4 Signed ref snapshots (sigrefs)

Each author publishes a chain of snapshots. A snapshot is a Git commit whose tree contains two files: `refs` (sorted lines of `<oid> <refname>`) and `signature`. The commit parent is the previous snapshot, so the chain is append-only and replicates with ordinary Git fetch.

```rust
pub const DOMAIN_SIGREFS: &[u8] = b"oggit/sigrefs/v1\0";

#[derive(Serialize, Deserialize)]
pub struct SigrefsBody {
    pub repo: RepoId,
    pub author: AuthorId,
    pub seq: u64,                          // strictly increasing
    pub parent: Option<[u8; 32]>,          // blake3 of previous canonical body
    pub refs: BTreeMap<String, Oid>,       // refname -> tip
}

pub enum Reject { NotMember, BadSignature, Stale, BrokenChain, WrongRepo }

pub fn accept_sigrefs(
    repo: &RepoId,
    current: Option<&SigrefsBody>,
    new: &SigrefsBody,
    sig: &Signature,
    circle: &CircleState,
) -> Result<(), Reject> {
    if &new.repo != repo { return Err(Reject::WrongRepo); }
    if !circle.may_publish(&new.author) { return Err(Reject::NotMember); }
    let bytes = canonical_bytes(new);
    if !verify(&new.author, DOMAIN_SIGREFS, &bytes, sig) { return Err(Reject::BadSignature); }
    if let Some(cur) = current {
        if new.seq <= cur.seq { return Err(Reject::Stale); }
        // The chain must link back to what we already hold; otherwise the peer is
        // rewriting history or we are missing intermediate snapshots (fetch them).
        if new.parent != Some(hash_body(cur)) && new.seq == cur.seq + 1 {
            return Err(Reject::BrokenChain);
```

```
        }
    }
    Ok(())
}
```

Listing 4: Snapshot body and acceptance check.

### 5.5 Circle log

Membership is itself a signed, append-only log stored under `refs/og/circle`. Entries: `AddMember(author, role)`, `RemoveMember`, `SetThreshold`, `RotateKey`. A delegate-changing entry is valid only if signed by at least `threshold` current delegates. The log is a pure reducer: `CircleState = fold(log)`; this makes it easy to property-test.

### 5.6 Petnames

Names are *local*. `og alias add website og://...` stores a mapping in your config. The genesis `name` field is only a suggested label shown when you first clone. Optional signed name records (author → label) can later be published to the DHT as a convenience, never as an authority.

## 6 Network Layer

### 6.1 Transport stack

- **QUIC (UDP)** as the primary transport: multiplexed streams without head-of-line blocking, fast handshakes, connection migration, built-in encryption. TCP + Noise + Yamux as fallback for restrictive networks.
- **Mutual authentication.** The transport handshake proves possession of the node key. The node-to-author certificate binds it to a developer identity.
- **Application protocols** as separate stream protocols, negotiated by ID (Appendix A).

### 6.2 Discovery layers (from nearest to farthest)

| Layer | Mechanism | Purpose |
|---|---|---|
| 1. Hints | Multiaddrs embedded in `og://` and invite links. | First contact; works with zero infrastructure. |
| 2. LAN | mDNS. | Offices, hackathons, planes, offline labs. |
| 3. Circle peers | Known members' nodes from the circle log and prior sessions. | Warm start. |
| 4. Peer exchange | Peers share peers they know for a repo. | Resilience when bootstrap nodes vanish. |
| 5. DHT | Kademlia *provider records* keyed by blake3("oggit/provider/v1"‖RepoId). | Find any seed of a repo knowing only its ID. |
| 6. Bootstrap list | Configurable list of well-known nodes (DNS-addressed). | Join the DHT. Anyone can run one. |

### 6.3 NAT traversal

Most developers sit behind NAT. Use the standard libp2p combination: **AutoNAT** to learn reachability, **circuit relay v2** as a fallback path, and **DCUtR** to upgrade relayed connections to direct ones by hole punching. Any publicly reachable `ogd` may enable `relay = true` to help others. Relayed connections are bandwidth-limited by design; large transfers should wait for the direct upgrade.

### 6.4 Gossip

One GossipSub topic per repository: `og/v1/repo/<hex repo-id>`. A node subscribes only to repositories it seeds or follows, so there is no global firehose. Messages are tiny and signed:

```
#[derive(Serialize, Deserialize)]
pub enum Gossip {
    /// "Author X now has sigrefs at sequence N with head H."
    RefsAnnounce { repo: RepoId, author: AuthorId, seq: u64, head: Oid, sig: Signature },
    /// Compact summary of what a seed holds (bounded size, sampled for big nodes).
    Inventory { repos: Vec<RepoId> },
    /// "I am looking for a seed of this repo."
    Want { repo: RepoId },
}
```

Listing 5: Gossip messages.

### 6.5 Libp2p behaviour sketch

```
use libp2p::{autonat, dcutr, gossipsub, identify, kad, mdns, relay, swarm::NetworkBehaviour};

#[derive(NetworkBehaviour)]
pub struct OgBehaviour {
    pub kad: kad::Behaviour<kad::store::MemoryStore>,
    pub gossip: gossipsub::Behaviour,
    pub identify: identify::Behaviour,
    pub mdns: mdns::tokio::Behaviour,
    pub autonat: autonat::Behaviour,
    pub relay_client: relay::client::Behaviour,
    pub dcutr: dcutr::Behaviour,
    pub streams: libp2p_stream::Behaviour,   // raw streams for /og/sync/1
}

pub fn build_swarm(keypair: libp2p::identity::Keypair) -> anyhow::Result<libp2p::Swarm<
    OgBehaviour>> {
    let mut swarm = libp2p::SwarmBuilder::with_existing_identity(keypair)
        .with_tokio()
        .with_quic()
        .with_dns()?
        .with_relay_client(libp2p::noise::Config::new, libp2p::yamux::Config::default)?
        .with_behaviour(|key, relay| OgBehaviour::new(key, relay))?
        .with_swarm_config(|c| c.with_idle_connection_timeout(std::time::Duration::from_secs(60))
    )
        .build();
    swarm.listen_on("/ip4/0.0.0.0/udp/0/quic-v1".parse()?)?;
    Ok(swarm)
}

// Announce that we seed a repo:
// swarm.behaviour_mut().kad.start_providing(provider_key(&repo_id))?;
```

Listing 6: Composing the swarm (API sketch; adapt to your libp2p version).

# 7 Sync and Replication Engine

## 7.1 Two-stage strategy

**Do not write a pack protocol in version 1**

**Stage A (MVP):** tunnel Git's *own* wire protocol over the libp2p stream. The serving side runs `git upload-pack` on the mirror; the fetching side runs `git fetch` with an `ext::` transport that pipes stdio into the stream. You get packfiles, deltas, negotiation, and partial clone for free, and you are Git-compatible by construction.
**Stage B (performance):** replace the tunnel with a native protocol that fetches from many peers in parallel, caches packs, and uses verified chunk streaming for large objects.

## 7.2 Stage A details

1. The serving daemon receives a stream with header `Open{repo, service=UploadPack}`.
2. It checks authorization (public repo, or the remote node's author is in the circle). If denied, it closes with an error code.
3. It spawns `git upload-pack` on `repos/<id>` and copies bytes both ways between the process and the stream. Mirror config: `uploadpack.allowFilter=true`, `uploadpack.allowAnySHA1InWant=true`, hidden refs for private areas.
4. The fetching daemon runs, for each followed author, a fetch of `+refs/namespaces/<author>/*:refs/namespaces/<author>/*` through `ext::og-pipe`.
5. After the fetch, the engine verifies sigrefs (Section 5) *before* making new refs visible to the canonical computation.

## 7.3 Sync loop

```
on RefsAnnounce(repo, author, seq, head) from any peer:
    if not following(repo, author):            drop
    if seq <= local_seq(repo, author):         drop            # stale or duplicate
    if announce.sig invalid:                   penalize peer; drop
    enqueue FetchJob(repo, author, peers=[sender, ...providers(repo)])

FetchJob:
    for peer in rank(peers):                    # fastest and most reliable first
        stream = connect(peer, "/og/sync/1")
        git fetch via stream into temporary quarantine area
        if fetch failed: penalize peer; continue
        verify sigrefs chain from local_seq+1 .. N
        verify every announced ref tip exists and is reachable
        run fsck / object-count / size limits
        atomically move refs from quarantine into mirror
        update index (seq), re-announce to topic, recompute canonical refs
        return
    schedule retry with exponential backoff and jitter
```

Listing 7: Sync algorithm (pseudocode).

## 7.4 Stage B: multi-peer parallel fetch

1. **Skeleton first.** Fetch only commits using a treeless/blobless filter (`--filter=tree:0`) from one fast peer. This yields the full commit graph cheaply.

2. **Partition.** Split the commit graph into disjoint ranges (by topological slices). Each range is expressed as `want X have Y`.
3. **Assign** ranges to different peers; hedge the slowest range by asking a second peer after a timeout.
4. **Ingest** each returned pack through `index-pack --fix-thin --strict`; Git's object hashes provide integrity, so a lying peer is detected immediately.
5. **Large objects** use the chunk store (below), with rarest-first scheduling like BitTorrent.

### 7.5 Large Object Store (LOS)

Files above a threshold (default 1 MiB) are stored as chunks: content-defined chunking (FastCDC, average 256 KiB), each chunk hashed with BLAKE3, a manifest listing chunk hashes. Git holds a small pointer blob:

```
oggit-large v1
manifest blake3:9f2c...e1
size 734003200
```

Chunks are fetched from *any* peer that has them using `/og/chunks/1`; BLAKE3's tree structure allows verified streaming and resumable downloads. Deduplication across versions and repositories is automatic.

### 7.6 Seeding policy

| Policy | Behavior |
|---|---|
| `none` | Fetch only; serve nothing (leech mode; discouraged). |
| `followed` | Seed repositories you follow or contribute to (default). |
| `allowlist` | Seed only listed repositories (good for company infrastructure). |
| `all` | Seed everything requested (community seed nodes). Disk and bandwidth quotas apply. |

*Pinning.* `og seed add <repo>` guarantees the node keeps a repository and re-announces it periodically. Run a pinning node on a cheap VPS or home server for high availability; any member can do this, so no one is special.

## 8 Developer Circles and Access Control

A *circle* is the group of people around a project. It answers requirement 4 with three ideas: roles, invites, and quorum.

### 8.1 Roles

| Role | Capabilities |
|---|---|
| **Delegate** | Maintainers. Their namespaces determine the canonical branches. Can edit the circle (threshold signatures). |
| **Member** | Trusted contributors. Their namespaces are auto-followed and seeded by the circle; can open issues and patches; can read private repos. |
| **Guest** | Read-only access to a private repository. |
| **Anyone** | On public repositories, anyone can fetch and anyone can publish in *their own* namespace; it only affects the project if a delegate merges it. |

### 8.2 Invite flow (target: under 60 seconds)

1. Maintainer: `og circle invite --role member --ttl 48h` prints `og://invite/<token>` and a QR code.
2. The token is a signed capability: repo ID, role, expiry, one-time nonce, inviter, and seed multiaddr hints.
3. New member: `og clone og://invite/<token>`. Their node connects to the hinted seed, presents the token together with its author public key.
4. The inviter's node (or any delegate node holding the nonce) verifies the token, marks the nonce as spent, and appends `AddMember` to the circle log, signed by a delegate.
5. The new member's node fetches the log and the repository. Done.

```rust
#[derive(Serialize, Deserialize)]
pub struct InviteBody {
    pub repo: RepoId,
    pub role: Role,
    pub inviter: AuthorId,
    pub expires_unix: u64,
    pub nonce: [u8; 16],
    pub seed_hints: Vec<String>,   // multiaddr strings
}
#[derive(Serialize, Deserialize)]
pub struct Invite { pub body: InviteBody, pub sig: Signature }
// Encode: base58(cbor(Invite)) -> og://invite/<base58>
```

Listing 8: Invite capability.

### 8.3 Canonical branches by quorum

For each branch (for example `refs/heads/main`), ogGIT computes the canonical tip from the delegates' namespaces:

```rust
/// tips: each delegate's tip for this branch (deduplicated).
pub fn canonical_tip(g: &CommitGraph, tips: &[Oid], threshold: usize) -> Option<Oid> {
    // A candidate is any tip that at least `threshold` delegate tips contain
    // (i.e. the tip is an ancestor of, or equal to, their own tip).
    let candidates: Vec<Oid> = tips.iter().copied()
        .filter(|&c| tips.iter().filter(|&&t| g.is_ancestor_or_eq(c, t)).count() >= threshold)
        .collect();

    // Keep only maximal candidates (no other candidate descends from them).
    let maximal: Vec<Oid> = candidates.iter().copied()
```

```
        .filter(|&c| !candidates.iter().any(|&d| d != c && g.is_ancestor_or_eq(c, d)))
        .collect();

    // Exactly one maximal candidate: that is the canonical tip.
    // Zero or several: the delegates have diverged; keep the previous canonical tip.
    if maximal.len() == 1 { Some(maximal[0]) } else { None }
}
```

Listing 9: Quorum rule: the maximal commit that at least ‘threshold‘ delegates have reached.

Requiring `threshold` > half of the delegates prevents two disjoint groups from each claiming a quorum. A delegate that force-pushes simply stops “voting” for the old history; the canonical tip does not move until enough delegates agree on a descendant. With one delegate the rule degenerates to “that delegate’s tip”.

### 8.4 Private repositories

Two complementary layers:

1. **Access control at the connection.** The stream header is authenticated by the node key, mapped to an author via the delegation certificate. A private repo’s streams are served only to circle members. Private repos are not announced to the public DHT in cleartext (provider key is derived from a repo secret known to members).
2. **End-to-end encryption for untrusted seeds (optional).** A symmetric repo key encrypts packs at rest and in transit to a pinning node that is not a member. The key is wrapped for each member using X25519 (derived from Ed25519 keys) and stored in the circle log. Removing a member rotates the key; new objects use the new key. (Old data the member already downloaded cannot be recalled.)

### 8.5 Key rotation and recovery

- **Rotation:** the old author key signs a `RotateKey(new)` entry; delegates countersign.
- **Loss:** delegates (threshold) can replace a lost member key, and a lost *delegate* key can be replaced by the remaining quorum. This is social recovery: no central reset exists, by design.
- **Single-delegate repos** have no recovery; `og init` warns and offers a recovery-key export.

## 9 Collaboration Objects: Issues, Patches, Reviews

Code hosting without issues and review is not very helpful. ogGIT stores them in the same repository so they replicate, work offline, and stay signed.

- Each object (issue, patch, comment thread) is a DAG of *operations*. Each operation is a Git commit containing a small CRDT op, signed by its author, stored under `refs/og/cobs/<type>/<id>`.
- State = deterministic merge of all operations from all authors (ordered by the DAG, ties broken by op hash). Concurrent edits converge without a server.
- Use an existing CRDT library (`automerge`) or a custom design: last-writer-wins registers for fields (title, status), a sequence CRDT (RGA) for text, add-wins sets for labels.
- **Patch** = a series of *revisions*; each revision is a commit range plus a base. Reviews and comments attach to revisions.

- **Merge** = a delegate updates their branch to include the patch; the quorum rule then moves canonical `main`. The patch object records the merge automatically by observing ancestry.
- Only operations authored by members (for the relevant action) are applied; others are retained but marked "untrusted" and hidden by default.

# 10 Git Integration

## 10.1 Remote helper

Git calls `git-remote-og` whenever a remote URL starts with `og://`. The helper speaks Git's simple line protocol on stdin and stdout, and talks to `ogd` over the Unix socket. Because the helper only touches the *local* mirror, `git fetch` and `git push` feel instant and work offline; `ogd` propagates changes in the background.

```
// usage by git: git-remote-og <remote-name> <url>
use std::io::{self, BufRead, Write};

fn main() -> anyhow::Result<()> {
    let mut args = std::env::args().skip(1);
    let _remote = args.next();
    let url = args.next().ok_or_else(|| anyhow::anyhow!("missing url"))?;
    let rid = RepoId::from_url(&url)?;                 // og://<repo-id> or og://invite/...
    let rpc = OgRpc::connect_default()?;               // starts ogd if it is not running
    let mirror = rpc.ensure_repo(&rid)?;               // clones/syncs in background, returns
    path

    let stdin = io::stdin();
    let mut out = io::stdout().lock();
    let (mut fetches, mut pushes): (Vec<String>, Vec<String>) = (vec![], vec![]);

    for line in stdin.lock().lines() {
        let line = line?;
        let mut it = line.splitn(2, ' ');
        match (it.next().unwrap_or(""), it.next()) {
            ("capabilities", _) => { writeln!(out, "fetch\npush\noption\n")?; }
            ("list", _) => {
                // Canonical refs computed by ogd (quorum rule), plus HEAD.
                for (oid, name) in rpc.canonical_refs(&rid)? { writeln!(out, "{oid} {name}")?; }
                writeln!(out)?;
            }
            ("option", _) => { writeln!(out, "unsupported")?; }
            ("fetch", Some(a)) => fetches.push(a.to_string()),    // "<sha> <refname>"
            ("push",  Some(a)) => pushes.push(a.to_string()),     // "[+]<src>:<dst>"
            ("", _) => {                                          // blank line ends a batch
                if !fetches.is_empty() {
                    run_fetch(&mirror, &fetches)?;                // git fetch <mirror> <sha>...
                    writeln!(out)?;
                    fetches.clear();
                } else if !pushes.is_empty() {
                    for status in run_push(&rpc, &rid, &mirror, &pushes)? { writeln!(out, "{
    status}")?; }
                    writeln!(out)?;
                    pushes.clear();
                } else { break; }
            }
            _ => {}
        }
        out.flush()?;
    }
    Ok(())
```

```
}
```

Listing 10: git-remote-og main loop (sketch).

**Fetch:** run `git fetch --no-tags --no-write-fetch-head <mirror-path> <sha>...` inside the user's repository (Git exports `GIT_DIR` to the helper). The mirror allows any-SHA wants.

**Push:** for each refspec, run `git push <mirror-path> <src>:refs/namespaces/<me>/<dst>`, then ask `ogd` to create a new sigrefs snapshot and announce it. Reply `ok <dst>` or `error <dst> <reason>`.

**Pull requests without a web service:** `og patch open` pushes `refs/og/patches/...` in your namespace and creates a patch object; reviewers fetch it like any branch.

### 10.2 URL forms

| URL | Meaning |
|---|---|
| `og://<repo-id>` | Clone/fetch via DHT discovery. |
| `og://<repo-id>?seed=<multiaddr>` | Same, with a seed hint for fast first contact. |
| `og://invite/<token>` | Redeem an invite, then clone. |
| `og:<alias>` | Local petname. |

### 10.3 Compatibility extras

- **Mirror out:** `og mirror add github <url>` pushes canonical branches to a normal Git host for discoverability and CI. This is the main adoption bridge.
- **Mirror in:** import an existing repo with `og init` inside it; history is preserved untouched.
- **Signed commits:** keep working normally; ogGIT authentication is at the ref-snapshot level, independent of commit signing.

## 11 CLI and User Experience

### 11.1 Sixty-second tour

```
# Install (single static binary; also brew, apt, nix, cargo)
cargo install og-cli

# One-time setup: creates keys, starts the node, discovers LAN peers
og init

# Publish an existing project
cd myproject
og repo create --name myproject      # genesis + adds git remote "og"
git push og main                     # normal git, instant (local mirror)

# Invite a teammate
og circle invite --role member       # prints og://invite/... and a QR code

# Teammate
og clone og://invite/2xQ...          # joins the circle and clones

# Day to day
git pull og main
og patch open -m "Fix parser"        # review without leaving the terminal
```

```
og issue list
og status                              # peers, seeds, sync state, disk use
```

Listing 11: The experience to aim for.

### 11.2 Command reference (v1)

| Command | Purpose |
|---|---|
| `og init` | Create keys, config, start the node. |
| `og node [start\|stop\|install-service]` | Control the daemon; install as systemd/launchd service. |
| `og repo create\|list\|info` | Manage repositories. |
| `og clone <url>` | Clone via DHT or invite. |
| `og circle invite\|list\|add\|remove\|threshold` | Manage the developer circle. |
| `og follow\|unfollow <author>` | Choose whose namespaces to replicate. |
| `og seed add\|rm\|list` | Pin repositories on this node. |
| `og patch\|issue ...` | Collaboration objects. |
| `og mirror add\|sync` | Bridge to conventional Git hosts. |
| `og alias add\|rm` | Local petnames. |
| `og doctor` | Diagnose NAT, connectivity, disk, permissions. |
| `og status` | Live state of peers and sync. |

### 11.3 Usability rules

- Every error message says what failed, why, and the exact next command to run.
- `og doctor` must make network problems self-explanatory ("You are behind symmetric NAT; relayed connections only. Run a relay or open UDP port...").
- The daemon starts on demand; users never need to know it exists.
- Defaults are safe: passphrase on keys, conservative quotas, private repos not announced.
- Progress output shows *which peers* data is coming from; this builds intuition for the model.

## 12 Daemon Internals

### 12.1 Actor structure

`ogd` is a set of tokio tasks that communicate through bounded channels. No shared mutable state except through the index database.

| Task | Responsibility |
|---|---|
| `Swarm` | Owns the libp2p swarm; translates network events into internal events and commands into network actions. |
| `Router` | Dispatches events: gossip → Sync; incoming streams → Server workers. |
| `Sync` | Maintains fetch jobs, quarantine, verification, retries, scheduling. |
| `Server` | Serves upload-pack and chunk requests after authorization and rate limiting. |
| `Circle` | Applies circle log, answers "may X do Y?", computes canonical refs. |
| `Policy` | Seeding, quotas, banning, peer scoring. |
| `Rpc` | Unix-socket JSON-RPC for CLI, remote helper, TUI, web UI. |

```rust
#[tokio::main]
async fn main() -> anyhow::Result<()> {
    let cfg = Config::load()?;
    let store = Store::open(&cfg)?;                    // redb + repo registry
    let (net_tx, mut net_rx) = tokio::sync::mpsc::channel(1024);
    let (cmd_tx, cmd_rx) = tokio::sync::mpsc::channel(1024);

    tokio::spawn(swarm_task(cfg.clone(), net_tx, cmd_rx));
    let sync = SyncEngine::new(store.clone(), cmd_tx.clone());
    let server = Server::new(store.clone());
    let rpc = rpc::serve(cfg.socket_path(), store.clone(), sync.handle());

    loop {
        tokio::select! {
            Some(ev) = net_rx.recv() => match ev {
                NetEvent::Gossip { from, msg } => sync.on_gossip(from, msg).await,
                NetEvent::InboundStream { peer, stream } => { server.spawn_session(peer, stream);
 }
                NetEvent::Connected(p) | NetEvent::Disconnected(p) => sync.on_peer_change(p).
    await,
            },
            _ = tokio::signal::ctrl_c() => break,
            r = &mut rpc => { r?; }
        }
    }
    store.flush()?;
    Ok(())
}
```

Listing 12: Daemon main loop (sketch).

## 12.2 Rules for staying fast and correct

- Never block the async executor: pack building, `index-pack`, fsck and graph walks run in `spawn_blocking` or a dedicated thread pool.
- Every channel is bounded; every external input has a size limit and a timeout.
- All ref updates happen in a quarantine area and are applied atomically after verification.
- One repository-level lock for writes; unlimited concurrent readers.

## 12.3 Local APIs and UI

- JSON-RPC over the Unix socket (named pipe on Windows) is the single control surface for all clients.

- The web UI is served from `127.0.0.1` only; it shows commits, diffs, issues, patches, circle, and peer map. A read-only public gateway mode is optional and off by default.
- A `ratatui` TUI offers fast review and issue handling over SSH.

# 13 Performance Engineering

## 13.1 Where time goes in a clone

Connect and discover → negotiate → build pack (CPU, disk) → transfer (network) → index and verify (CPU, disk). Each stage has a known remedy:

| Stage | Technique |
|---|---|
| Connect | Long-lived daemon keeps warm connections; QUIC multiplexing; try hinted peers and LAN first; race several candidates ("happy eyeballs"). |
| Negotiate | Commit-graph files with changed-path Bloom filters; reachability *bitmaps* on seeds (`git repack -adb`); multi-pack-index. |
| Build pack | Pack cache keyed by (wants, haves); delta islands; reuse existing deltas; `pack.threads` tuned; precomputed "bundle" of the default branch on seeds. |
| Transfer | Parallel multi-peer ranges (Section 6.5); hedged requests; rarest-first chunks; large flow-control windows; pacing to avoid bufferbloat. |
| Index/verify | Streaming `index-pack` while receiving; mmap; bounded parallelism; `rayon` for independent work. |
| Large files | Chunk dedup (FastCDC + BLAKE3); verified streaming; resume after disconnect. |

## 13.2 Resource budgets (targets to validate, not promises)

- CLI cold start under 20 ms; `git push og` on a small change returns in under 100 ms (local mirror only).
- Idle daemon under 50 MB resident memory and near-zero CPU.
- Seed node: saturate a 1 Gbit/s link on large clones with proper bitmaps; memory bounded by configured pack-cache size.
- LAN clone limited by disk or network, not by ogGIT.

## 13.3 Benchmarking plan

1. Corpus: small (10 MB), medium (500 MB), large monorepo (several GB), binary-heavy repo.
2. Scenarios: first clone, incremental fetch, push of 1 commit, push of 100 commits, clone from 1 / 3 / 10 peers, clone over relay, clone with 150 ms RTT and 1% loss (`tc netem`).
3. Baselines: plain `git` over SSH/HTTPS to a hosted service on the same link; `git` over LAN to a bare repo.
4. Track results per commit in CI; fail the build on regressions beyond a threshold.

## 14 Security and Threat Model

| Threat | Impact | Mitigation |
|---|---|---|
| Forged updates | Fake commits appear official. | Signed sigrefs; acceptance only from circle members; canonical state only from delegates. |
| Rollback / replay | A malicious seed serves an old state. | Monotonic sequence numbers; remember highest seen; query several peers; periodic gossip freshness. |
| Malicious packfile | Zip/delta bombs, resource exhaustion. | Quarantine; limits on size, object count, delta depth, inflate ratio; `index-pack --strict`; fsck. |
| Sybil / spam | Flooding with junk repos or announcements. | Repo-scoped trust; subscribe only to wanted topics; per-peer rate limits and scoring; announcement size caps. |
| Eclipse attack | Victim sees only attacker's peers. | Multiple bootstrap sources; peer diversity (IP/ASN buckets); circle peers pinned in routing table; disjoint-path DHT lookups. |
| DoS on seeds | Bandwidth/CPU exhaustion. | Per-peer and global rate limits; cost accounting on requests; pack-cache; connection limits; optional allowlist mode. |
| Key theft | Attacker publishes as the author. | Encrypted key file; hardware keys; delegation certificates per device; revocation through circle log; threshold for delegates. |
| Privacy leak | IP addresses visible to peers. | Document clearly; option to disable announcing addresses; relay-only mode; Tor/I2P transport in a later phase. |
| Supply-chain | Malicious dependency in ogGIT itself. | `cargo-deny`, `cargo-audit`, vendored lockfile, reproducible builds, signed releases. |
| Code execution bugs | Remote exploit via parser. | Rust; fuzz all decoders; no `unsafe` outside audited modules; sandbox helper processes (seccomp / landlock) where possible. |
| Unwanted content | Legal or ethical issues for seeds. | Seeding is opt-in per repository; local blocklists; seeds are not obliged to host anything. |

**Limits you should state publicly**

Decentralization does not give deletion, perfect privacy, or guaranteed availability. Revoking access does not erase what someone already downloaded. A repository survives only while somebody seeds it. Say so in the documentation; trust is built on honesty about these limits.

# 15 Code Skeleton

## 15.1 Workspace layout

```
oggit/
  Cargo.toml
  crates/
    og-core/        # ids, genesis, sigrefs, circle log reducer, invites, quorum
    (pure logic, no I/O)
    og-store/       # bare repo management, gix/git wrappers, redb index, chunk
    store
    og-net/         # libp2p swarm, protocols, gossip, discovery, relay config
    og-sync/        # fetch jobs, quarantine, verification, scheduling
    og-cob/         # CRDT collaboration objects
    og-daemon/      # ogd binary: wires everything, RPC server
    og-cli/         # og binary
    git-remote-og/  # git remote helper binary
    og-testkit/     # in-process multi-node simulator, fixtures, fuzz targets
  docs/
  fuzz/
  benches/
```

Listing 13: Cargo workspace.

Keep `og-core` free of I/O and async: it should be a deterministic library you can test exhaustively and fuzz heavily. All security-critical rules (what is valid, who may do what) live there.

## 15.2 Workspace manifest

```
[workspace]
resolver = "2"
members = ["crates/*"]

[workspace.dependencies]
tokio = { version = "1", features = ["full"] }
libp2p = { version = "0.55", features = [
  "tokio", "quic", "dns", "noise", "yamux", "kad", "gossipsub",
  "identify", "mdns", "relay", "dcutr", "autonat", "macros", "ed25519"
] }
libp2p-stream = "0.3"
gix = "0.7"
ed25519-dalek = { version = "2", features = ["rand_core", "serde"] }
blake3 = "1"
serde = { version = "1", features = ["derive"] }
ciborium = "0.2"
redb = "2"
fastcdc = "3"
automerge = "0.5"
chacha20poly1305 = "0.10"
x25519-dalek = "2"
argon2 = "0.5"
clap = { version = "4", features = ["derive"] }
anyhow = "1"
thiserror = "1"
tracing = "0.1"
proptest = "1"

[profile.release]
lto = "fat"
codegen-units = 1
panic = "abort"
```

```
strip = true
```

Listing 14: Root Cargo.toml (versions indicative; pin at project start).

### 15.3 Build order inside the code

1. `og-core`: types, canonical encoding, signature domains, sigrefs acceptance, circle reducer, quorum function, invites. Test heavily.
2. `og-store`: create bare mirror, namespace refs, read ref tips, ancestry queries, atomic ref updates from quarantine.
3. `og-net`: swarm with QUIC, identify, mDNS; raw stream protocol; then Kademlia and GossipSub; then relay/DCUtR/AutoNAT.
4. `og-sync`: Stage A tunnel, verification pipeline, retry and backoff.
5. `og-daemon`, `og-cli`, `git-remote-og`: integration.
6. `og-cob`, UI, large objects, Stage B fetch.

## 16 Testing and Quality

| Layer | Approach |
|---|---|
| Pure logic | Unit tests and `proptest`: "any sequence of valid circle entries yields the same state regardless of merge order", "canonical tip never moves backwards", "invalid signatures are never accepted". |
| Wire formats | Round-trip tests; cross-version fixtures; `cargo-fuzz` on every decoder and on pack ingestion. |
| Single-node | Real `git` scripts against `git-remote-og`: clone, fetch, push, force-push, delete branch, tags, submodules, shallow and partial clone. |
| Multi-node | `og-testkit` spawns $N$ in-process nodes on in-memory or loopback transports; scenario scripts: late joiner, offline then reconnect, partition and heal, malicious seed, delegate disagreement. |
| Network realism | Linux network namespaces + `tc netem` for latency, loss, NAT types; Docker Compose scenarios in CI. |
| Chaos | Randomly kill, restart and partition nodes during sync; verify invariants (no corrupted refs, convergence after healing). |
| Compatibility | Run subsets of Git's own test scripts against a mirror; verify that a plain `git clone` of a mirror path works. |
| Security | Threat-model review per release; dependency audits; external audit before the 1.0 label. |

### 16.1 Invariants to assert everywhere

1. A ref becomes visible only after its sigrefs were verified.
2. The accepted sequence number for an author never decreases.
3. Canonical refs depend only on delegates' verified data and the circle state.
4. No network input can cause unbounded memory, disk, or CPU use.
5. After a partition heals, all nodes with the same inputs compute the same canonical state.

# 17 Roadmap

Durations are rough estimates for a small team (2–3 engineers). Treat them as planning aids, not commitments.

| Phase | Time | Deliverables | Exit criteria |
|---|---|---|---|
| 0. Spike | 2–3 wk | Two nodes exchange a repo via libp2p stream tunnel with `git upload-pack`. | `git clone` works between two laptops on LAN. |
| 1. Local-first MVP | 6–8 wk | Keys, genesis, per-author namespaces, sigrefs, remote helper, mirror, direct-connect and mDNS sync. | Two developers collaborate on a LAN and via direct address; all verification in place. |
| 2. Internet P2P | 6–8 wk | Kademlia providers, GossipSub, NAT traversal (AutoNAT, relay, DCUtR), seeding policies, quotas, `og doctor`. | Clone across home networks and via relay; bootstrap node outage does not break existing swarms. |
| 3. Circles | 4–6 wk | Circle log, roles, invite links, follow lists, quorum canonical refs, key rotation, private repos with access control. | 60-second onboarding test passes; delegate-disagreement scenarios behave. |
| 4. Collaboration | 6–8 wk | Issues, patches, reviews as CRDT objects; TUI; local web UI; GitHub/GitLab mirror-out. | Project dogfoods ogGIT for its own development. |
| 5. Speed | 6–8 wk | Stage B multi-peer fetch, pack cache, bitmaps on seeds, large object store, benchmarks in CI. | Published benchmark results against hosted Git baseline. |
| 6. Hardening | 6–10 wk | Fuzzing, chaos tests, security review, packaging (brew, apt, nix, Docker), docs, signed releases. | External audit completed; 1.0 release candidate. |
| 7. Beyond | ongoing | Tor/I2P transport, SHA-256 repos, hardware keys, optional E2E-encrypted pinning, federation of web gateways. | Driven by user needs. |

# 18 Risks and Open Questions

1. **Cold-start network.** A P2P network with two users is just two computers. Mitigation: LAN and invite hints make small teams work on day one; run community seed nodes; mirror-out to conventional hosts for discoverability.
2. **Availability expectations.** Developers expect "push and it is online". Mitigation: circle auto-seeding, cheap always-on pinning nodes, clear status in `og status` ("3 seeds hold your latest push").
3. **Hash function transition.** Git is moving toward SHA-256. Keep `Oid` abstract; protocol-level hashes already use BLAKE3.
4. **Windows support.** Named pipes, path semantics, and `ext::` transport behavior differ;

budget time for it.

5. **Moderation and legal.** Decentralized systems still face abuse. Provide local blocklists and per-node policy; avoid a global authority by design.
6. **Quorum UX.** Explain divergence in plain language and give tools to resolve it (`og canonical explain main`).
7. **Build vs. adopt.** Reassess at the end of Phase 1: if a mature system already covers most of your needs, contributing upstream may be the better investment.

# A Wire Protocol Summary

## A.1 Protocol identifiers

| ID | Purpose |
|---|---|
| `/og/sync/1` | Authenticated stream. First frame `Open{repo, service}`; `service` is `UploadPack`, `Refs`, or `Sigrefs`. After the header, raw bytes of the chosen service flow in both directions (Stage A). |
| `/og/chunks/1` | Request/response for large-object chunks: `Have{hashes}`, `Get{hash, range}`, verified-stream reply. |
| `/og/invite/1` | Redeem an invite: `Redeem{token, author_pubkey}` → `Accepted{log_head}` or error. |
| GossipSub topic | `og/v1/repo/<hex repo-id>`. |
| Kademlia key | blake3("oggit/provider/v1"‖repo-id). |

## A.2 Frame format for control messages

```
+--------+---------+--------+----------------+------------------+
| magic  | version |  type  | length (varint)| payload (postcard)|
| "OG" 2B|   1B    |   1B   |   1-4 bytes    |  <= 1 MiB         |
+--------+---------+--------+----------------+------------------+
```

## A.3 Error codes

| Code | Meaning |
|---|---|
| `NOT_FOUND` | Repository not held by this node. |
| `FORBIDDEN` | Peer's author is not permitted (private repo or banned). |
| `RATE_LIMITED` | Retry after the given delay. |
| `TOO_LARGE` | Request would exceed configured limits. |
| `BAD_REQUEST` | Malformed or unsupported message. |
| `BUSY` | Node temporarily overloaded; try another peer. |

# B Example Configuration

```
[node]
name = "alice-laptop"
listen = ["/ip4/0.0.0.0/udp/0/quic-v1", "/ip6/::/udp/0/quic-v1"]
bootstrap = [
```

```
  "/dnsaddr/bootstrap.example.org/p2p/<peer-id>",   # replace with real nodes
]
relay = false                   # set true on public servers to help others

[seeding]
policy = "followed"             # none | followed | allowlist | all
max_disk = "50GiB"
max_peers = 64

[limits]
max_pack_size = "2GiB"
max_objects_per_fetch = 5000000
max_delta_depth = 100
rate_limit_per_peer = "20MiB/s"

[privacy]
announce_addresses = true
transports = ["quic", "tcp"]    # "tor" later

[ui]
web = "127.0.0.1:7777"
tui_theme = "dark"
```

Listing 15: `~/.og/config.toml`. Bootstrap addresses are placeholders.

## C Week-One Spike (do this first)

The fastest way to retire the biggest technical risk is a throwaway prototype of Stage A.

1. Create a new Rust binary with `libp2p` (QUIC + mDNS + `libp2p-stream`). Print your PeerId.
2. On *serve* mode: accept streams on `/og/sync/1`; for each, spawn `git upload-pack /path/to/bare.git` and copy bytes both directions (`tokio::io::copy_bidirectional`).
3. On *fetch* mode: dial a PeerId; run `git -c protocol.ext.allow=always fetch "ext::og-pipe <peer>" main`, where `og-pipe` is a small helper that connects stdio to the stream (or have your binary listen on a local Unix socket that Git's `ext::` command connects to).
4. Run it on two machines on the same Wi-Fi. Clone a real repository. Time it against `git clone` over SSH.
5. Add sigrefs signing and verification for a single branch. You now have the heart of OGGIT.

If this works, the remaining phases are engineering, not research.

## D References and Further Reading

- Radicle (peer-to-peer code collaboration): https://radicle.xyz
- libp2p specifications and documentation: https://libp2p.io
- gitoxide (pure Rust Git): https://github.com/GitoxideLabs/gitoxide
- iroh (QUIC, hole punching, BLAKE3 blobs): https://iroh.computer
- BLAKE3 and verified streaming: https://github.com/BLAKE3-team/BLAKE3
- Git documentation: remote helpers (`gitremote-helpers`), namespaces (`gitnamespaces`), protocol v2 (`gitprotocol-v2`), partial clone, commit-graph and bitmaps.

- Kademlia: Maymounkov and Mazières, "Kademlia: A Peer-to-peer Information System Based on the XOR Metric" (2002).
- CRDTs: Shapiro et al., "Conflict-free Replicated Data Types" (2011); the Automerge project.
- Freenet: Clarke et al., "Freenet: A Distributed Anonymous Information Storage and Retrieval System" (2000).